Web Scraping ETL Pipeline

End-to-End Data Extraction, Transformation, and Loading Report

Prepared by Hassan Jumaa (Riq) · Data Analyst · riq-wq.github.io/Myportfolio

1. Executive Summary

This project demonstrates an end-to-end ETL (Extract, Transform, Load) pipeline built in a single Jupyter Notebook. It scrapes 1,000 book listings across 50 paginated pages from a live website (books.toscrape.com), cleans the data with Pandas, and loads the final dataset into a MySQL database. The pipeline handles real-world data quality issues including character encoding bugs, inconsistent text formatting, and dynamic pagination.

Key outcomes:

- **1,000 records** scraped from 50 pages (20 books per page) with zero null values after cleaning.
- **4 columns** extracted: title, price, availability, rating — all type-safe after transformation.
- **Dynamic pagination** — loop stops on HTTP 404 instead of hardcoding 50 pages.
- **Encoding bug fixed** — forced UTF-8 encoding prevents '£' from rendering as 'Â£'.
- **Rate-limited requests** — 1-second delay between pages for responsible scraping.
- **Loaded into MySQL** — cleaned data stored in `scraped_books` table via SQLAlchemy + PyMySQL.

2. Methodology & Pipeline

The analysis followed a three-stage ETL pipeline:

- **Extract (requests + BeautifulSoup):** Fetched each page from books.toscrape.com, parsed HTML with lxml, and collected title, price, availability, and rating from each book listing.
- **Transform (Pandas):** Stripped currency symbols, converted prices to float, mapped word-based ratings to integers, converted availability to boolean, and verified zero nulls.
- **Load (MySQL):** Connected via SQLAlchemy + PyMySQL using credentials from a .env file, and loaded the cleaned DataFrame into a MySQL table with if_exists='replace'.

3. Data Cleaning & Transformation

Cleaning steps applied to the raw scraped data:

Step	Action	Result
Encoding fix	response.encoding = 'utf-8'	Prevents '£' → 'Â£' corruption
Price parsing	str.replace('£', '') + astype(float)	51.77 (float64)
Availability	str.contains('In stock') → bool	True/False (bool)
Rating mapping	map({'One':1, ..., 'Five':5})	1-5 (int64)
Null check	df.isnull().sum()	0 nulls across all 4 columns
Type verification	df.info()	title: object, price: float64, rating: int64, in_stock: bool

Key data-quality observations:

- No duplicate rows — all 1,000 book titles are unique.
- No missing values — all 1,000 records have title, price, rating, and availability.
- Price range: £10.00 – £59.99 (average ~£35.00).
- Rating scale: 1–5 stars (mapped from word-based ratings).
- Availability: 100% in stock (all books available at scrape time).

4. Dynamic Pagination

The scraper uses a while True loop that increments the page number each iteration. The loop breaks when the response status code is not 200 (HTTP 404 when the page doesn't exist). This is more robust than hardcoding 50 pages — if the site adds more pages, the scraper will still find them, and if pages are removed, it stops gracefully.

Results: 50 pages scraped (20 books each) = 1,000 total records. Stopped at page 51 (404).

Rate limiting: time.sleep(1) between each page request — responsible scraping practice.

5. Sample Output

Title	Price	Rating	In Stock
A Light in the Attic	£51.77	3	True
Tipping the Velvet	£53.74	1	True
Soumission	£50.10	1	True
Sharp Objects	£47.82	4	True
Sapiens: A Brief History of Humankind	£54.23	5	True

6. Recommendations

- Add **error handling and retries** for failed requests (e.g., exponential backoff).
- Schedule the pipeline to run automatically (cron job or Airflow).
- Build a Power BI dashboard on top of the scraped_books table.
- Extend to a real e-commerce site with anti-bot handling.
- Add a **data-quality gate** to validate scraped data before loading to MySQL.
- Store data in a data warehouse (BigQuery, Snowflake) for production scale.

7. Conclusion

The ETL pipeline successfully scraped 1,000 book listings from books.toscrape.com, cleaned the data by fixing encoding bugs, converting currency strings to floats, mapping word-based ratings to integers, and converting availability text to booleans. The cleaned data was loaded into a MySQL database for further analysis. The pipeline demonstrates real-world data engineering skills including dynamic pagination, rate limiting, and type-safe transformations.

Data Source

books.toscrape.com — a practice website for web scraping (static HTML, no anti-bot measures).

Tools Used

Python · requests · BeautifulSoup4 · lxml · Pandas · SQLAlchemy · PyMySQL · MySQL